# CIVICPULSE HUB

## Grievance Management & Feedback Redressal System

## FULL DEVELOPMENT SPECIFICATION

Version 1.0 • Grievance workflow update • Four-member team

Purpose: define grievance workflows, roles, data, API contracts, security, and team responsibilities before implementation.

# Document Control

| Item | Decision |
|---|---|
| Project | CivicPulse Hub |
| Document version | 2.0 — Grievance workflow draft |
| Stage | Planning; implementation has not started |
| Primary goal | College project and presentation |
| Complexity | Intermediate |
| Core concept | Citizens lodge grievances; admins assign and set deadlines; officers resolve with proof; citizens rate outcomes. |
| Out of scope | Real government integration or claims of official action unless separately approved. |

Approval rule: all four members review this document. Changes to shared table names, API fields, status values, or authentication rules must be discussed and documented before code changes.

# 1. Project Overview

CivicPulse Hub is a role-based grievance management portal. Citizens register and verify their email, submit categorized complaints with optional before-photos and location details, and track progress. Administrators triage complaints, set priority and deadlines, and assign department officers. Officers submit resolution notes and after-photos. Citizens rate the result and leave feedback.

The system demonstrates transparent workflow and accountability for a college project. It is not connected to a real government department unless a future, separately approved integration is built. A displayed resolution means the assigned officer recorded completion; citizen feedback can challenge that result and reopen the complaint.

## 1.1 Objectives

- Support citizen, department officer, and administrator roles with role-based access control.
- Provide registration, email OTP verification, login, JWT authentication, and password reset.
- Allow citizens to lodge complaints with title, description, category, location, and before-image evidence.
- Allow admins to assign officers, set priorities and deadlines, and monitor workload.
- Allow officers to update their own priority where permitted and resolve complaints with notes and after-image proof.
- Collect citizen ratings and written feedback; automatically reopen a resolved complaint when the rating is 1 or 2.
- Provide workload, status, recent-complaint, and daily-submission analytics; Python supports optional trend analysis.
- Demonstrate secure full-stack development, auditability, and coordinated GitHub teamwork.

## 1.2 User roles

| Role | Permissions |
|---|---|
| Citizen | Register/verify/login; lodge complaints; view own history; submit feedback and ratings for resolved complaints. |
| Department officer | View assigned workload; update permitted officer priority; resolve assigned complaints with notes and after-image proof. |
| Administrator | Manage users/roles and departments; triage complaints; assign officers; set priority/deadline; view workload and system reports. |

# 2. Grievance Management Feature Scope

| ID | Feature | Description | Priority | Owner |
|---|---|---|---|---|
| F01 | Authentication | Register, login/logout, email OTP | Must | M2 |

| | | | | |
|---|---|---|---|---|
| | | verification, JWT authorization, forgot/reset password. | | |
| F02 | Citizen complaints | Create complaints with title, description, category/subcategory, location, and before-image. | Must | M1/M2/M3 |
| F03 | Admin triage | List complaints, set priority and deadline, assign eligible department officer. | Must | M1/M2 |
| F04 | Officer workflow | View assigned workload, update permitted officer priority, resolve with notes and after-image. | Must | M1/M2 |
| F05 | Citizen history | Show the citizen's own complaints and current workflow status. | Must | M1/M2 |
| F06 | Feedback and reopening | Collect 1–5 rating and text; ratings 1 or 2 reopen complaint to PENDING while preserving history. | Must | M1/M2/M3 |
| F07 | Workload analytics | Show counts by status, recent/daily submissions, and officer workload. | Must | M1/M2/M3 |
| F08 | Email verification | Send, verify, expire, and resend one-time email OTPs with rate limits. | Must | M2 |
| F09 | Resolution evidence | Store before/after image metadata and resolution notes. | Must | M1/M2/M3 |
| F10 | Audit history | Record assignment, deadline, priority, status, resolution, and reopening actions. | Must | M2/M3 |
| F11 | Spreadsheet export | Allow administrators to export authorized complaint/report data as XLSX. | Should | M1/M2 |
| F12 | Python analytics | Optional trend/workload analysis using bounded, non-sensitive data supplied by Express. | Should | M4/M2 |
| F13 | Role management | Administrator manages user activation and role assignment with audit records. | Must | M1/M2/M3 |
| F14 | Password reset | Issue expiring, single-use reset token and safely update password. | Must | M2/M3 |
| F15 | Independent module | Each member can | Must | All |

| | | | | |
|---|---|---|---|---|
| | tests | test their module without requiring the other modules to be running. | | |

Out of scope for the first release: integration with actual government systems, claims of official government action, live tracking of citizens, payments, and automatic verification that a physical repair truly occurred.

# 3. User Journeys

## 3.1 Register, verify, and lodge a grievance

1. Citizen registers with name, email, password, and any required role-specific fields.
2. The backend creates an OTP verification record and sends a code by email. The account cannot use protected citizen functions until verified.
3. After OTP verification and login, the citizen submits a complaint with title, description, category/subcategory, location, and an optional before-image.
4. Express validates the user, request, image, and role; stores the complaint as PENDING and records a status-history entry.
5. The citizen can view their own complaint history and the current status, priority, assigned officer when appropriate, and deadline.

## 3.2 Admin triage and officer assignment

6. User opens the map and sees pins inside the current map bounds.
7. The admin reviews pending complaints, confirms category and priority, assigns an eligible officer, and sets a resolution deadline.
8. The officer views assigned work, updates permitted officer priority, and records progress.
9. The officer resolves the complaint by providing resolution notes and an after-image.
10. The complaint changes to RESOLVED and stores the resolution timestamp, resolving officer, and evidence.

## 3.3 Resolution, citizen feedback, and reopening

11. The citizen submits a rating from 1 to 5 and optional written feedback for a resolved complaint.
12. If the rating is 1 or 2, the complaint returns to PENDING for administrative triage; the previous resolution and feedback remain in history.
13. If the rating is 3, 4, or 5, the complaint remains resolved and is eligible for finalization/archive according to the agreed policy.
14. Every status change records who performed it, when it happened, and an optional reason. Reopening does not erase prior evidence or audit history.

# 4. Four-Member Work Allocation

| Member | Ownership | Main deliverables | Dependencies |
|---|---|---|---|
| M1 — Frontend | React + Vite UI for citizen, officer, and admin portals. | OTP/login screens; complaint form/history; assignment/workload dashboards; resolution and feedback screens; validation and API client. | API contract from M2; shared fields/statuses from M3. |
| M2 — Backend | Node/Express APIs, authentication, authorization, workflow rules, email OTP, uploads, reports, and security. | REST API; JWT; role guards; complaint transitions; feedback/reopening; XLSX export; integration tests. | Schema from M3; API agreement with M1; analytics contract with M4. |
| M3 — Database | MySQL schema, | schema.sql, seed-dev.sql, | Approved fields and |

| | | | |
|---|---|---|---|
| | relationships, indexes, seed data, and migration/setup instructions. | test schema/data, ERD, constraints and repeatable local setup. | workflow rules from M1/M2/M4. |
| M4 — Python and QA | FastAPI trend/workload analysis, API contract tests, end-to-end plan, and integration coordination. | Python service; fixtures and tests; test report; integration checklist and reproducible bug reports. | Approved complaint/status fields and bounded analytics input from M2/M3. |

Each member tests their own module; M4 coordinates QA and analytics but does not carry all testing alone. M3 defines the schema with M2, and the team agrees on API and status contracts before implementation.

# 5. Technology Architecture

| Component | Technology | Responsibility | Proposed local address |
|---|---|---|---|
| Frontend | HTML, CSS, JavaScript, React + Vite | Citizen, officer, and admin portals; complaint forms, tracking, feedback, dashboards. | http://localhost:5173 |
| Backend | Node.js + Express | OTP/JWT authentication, RBAC, complaint workflow, assignments, deadlines, feedback, reports, uploads. | http://localhost:5000/api/v1 |
| Database | MySQL through XAMPP/phpMyAdmin | Users, OTP/reset records, complaints, attachments, feedback, history, departments, audit events. | 127.0.0.1:3306 |
| Python service | Python + FastAPI + Uvicorn | Optional trend/workload analysis only; core workflow does not depend on it. | http://127.0.0.1:8000 |
| Tools | VS Code, Git, GitHub | Development, collaboration, version control, tests, and pull requests. | N/A |

Use FastAPI for optional trend/workload analysis. React must never connect directly to MySQL. Python does not own core complaint writes; Express supplies bounded, non-sensitive data for analysis. Core grievance workflows must continue if Python is unavailable.

# 6. Database Specification

Database name: civicpulse_db. Use InnoDB and utf8mb4, foreign keys, indexes, unique constraints, and UTC timestamps. The source requirements define users and complaints; supporting tables below are proposed to safely implement OTP, attachments, feedback, history, departments, and auditability.

## 6.1 users

| Column | Type | Rules / purpose |
|---|---|---|
| user_id | BIGINT UNSIGNED | PK, AUTO_INCREMENT |
| name | VARCHAR(255) | NOT NULL; full name |
| email | VARCHAR(254) | NOT NULL, UNIQUE; login email |
| password_hash | VARCHAR(255) | NOT NULL; bcrypt/Argon2 hash only |
| role | VARCHAR(30) | NOT NULL; CITIZEN, OFFICER, ADMIN |
| department_id | BIGINT UNSIGNED | NULL for citizens/admins; FK departments for officers |
| active | TINYINT(1) | NOT NULL DEFAULT 1; account |

| | | |
|---|---|---|
| | | active state |
| verified | TINYINT(1) | NOT NULL DEFAULT 0; email OTP verification state |
| created_at | DATETIME | NOT NULL; UTC timestamp |
| updated_at | DATETIME | NOT NULL; UTC timestamp |

## 6.2 complaints

| Column | Type | Rules / purpose |
|---|---|---|
| complaint_id | BIGINT UNSIGNED | PK, AUTO_INCREMENT |
| title | VARCHAR(255) | NOT NULL |
| description | TEXT | NOT NULL |
| category | VARCHAR(100) | NOT NULL; controlled category/department classification |
| sub_category | VARCHAR(100) | NULL; e.g. pothole, water leak |
| location | VARCHAR(255) | NOT NULL or nullable according to team decision |
| image | VARCHAR(500) | NULL; before-image storage key/path |
| status | VARCHAR(20) | NOT NULL; PENDING, ASSIGNED, RESOLVED |
| priority | VARCHAR(20) | NOT NULL DEFAULT MEDIUM; LOW, MEDIUM, HIGH |
| officer_priority | VARCHAR(20) | NULL; officer-specific priority where allowed |
| user_id | BIGINT UNSIGNED | NOT NULL; FK users.user_id, citizen owner |
| officer_id | BIGINT UNSIGNED | NULL; FK users.user_id, assigned officer |
| deadline | DATE | NULL; admin-set resolution deadline |
| created_at | DATETIME | NOT NULL; UTC timestamp |
| resolved_image | VARCHAR(500) | NULL; after-image storage key/path |
| resolved_notes | TEXT | NULL; officer's work summary |
| resolved_at | DATETIME | NULL; resolution timestamp |
| resolved_by | BIGINT UNSIGNED | NULL; FK users.user_id, resolving officer |
| updated_at | DATETIME | NOT NULL; UTC timestamp |

## 6.3 complaint_attachments

| Column | Type | Rules / purpose |
|---|---|---|
| attachment_id | BIGINT UNSIGNED | PK, AUTO_INCREMENT |
| complaint_id | BIGINT UNSIGNED | NOT NULL; FK complaints.complaint_id |
| attachment_type | VARCHAR(20) | BEFORE or AFTER |
| file_path | VARCHAR(500) | NOT NULL; server-generated storage key/path |
| mime_type | VARCHAR(100) | Validated image content type |
| uploaded_by | BIGINT UNSIGNED | NOT NULL; FK users.user_id |
| created_at | DATETIME | NOT NULL; UTC timestamp |

## 6.4 complaint_feedback

| Column | Type | Rules / purpose |
|---|---|---|
| feedback_id | BIGINT UNSIGNED | PK, AUTO_INCREMENT |
| complaint_id | BIGINT UNSIGNED | NOT NULL; FK complaints.complaint_id |
| citizen_id | BIGINT UNSIGNED | NOT NULL; FK users.user_id |
| rating | TINYINT UNSIGNED | NOT NULL; 1–5; enforce check in backend and DB if supported |

| | | |
|---|---|---|
| feedback_text | TEXT | NULL; citizen review |
| resolution_cycle | INT UNSIGNED | NOT NULL; distinguishes feedback after each resolution |
| created_at | DATETIME | NOT NULL; UTC timestamp |

### 6.5 otp_verifications

| Column | Type | Rules / purpose |
|---|---|---|
| otp_id | BIGINT UNSIGNED | PK, AUTO_INCREMENT |
| user_id | BIGINT UNSIGNED | NOT NULL; FK users.user_id |
| otp_hash | VARCHAR(255) | NOT NULL; store hash, not raw OTP where practical |
| expires_at | DATETIME | NOT NULL |
| used_at | DATETIME | NULL; single-use token marker |
| attempt_count | INT UNSIGNED | NOT NULL DEFAULT 0; rate-limit verification attempts |
| created_at | DATETIME | NOT NULL; UTC timestamp |

### 6.6 password_reset_tokens

| Column | Type | Rules / purpose |
|---|---|---|
| reset_id | BIGINT UNSIGNED | PK, AUTO_INCREMENT |
| user_id | BIGINT UNSIGNED | NOT NULL; FK users.user_id |
| token_hash | VARCHAR(255) | NOT NULL; never store raw reset token |
| expires_at | DATETIME | NOT NULL |
| used_at | DATETIME | NULL; single-use token marker |
| created_at | DATETIME | NOT NULL; UTC timestamp |

### 6.7 complaint_status_history

| Column | Type | Rules / purpose |
|---|---|---|
| history_id | BIGINT UNSIGNED | PK, AUTO_INCREMENT |
| complaint_id | BIGINT UNSIGNED | NOT NULL; FK complaints.complaint_id |
| changed_by | BIGINT UNSIGNED | NOT NULL; FK users.user_id |
| old_status | VARCHAR(20) | NULL; previous status |
| new_status | VARCHAR(20) | NOT NULL; new status |
| reason | VARCHAR(500) | NULL; action reason or note |
| created_at | DATETIME | NOT NULL; UTC timestamp |

### 6.8 departments

| Column | Type | Rules / purpose |
|---|---|---|
| department_id | BIGINT UNSIGNED | PK, AUTO_INCREMENT |
| name | VARCHAR(255) | NOT NULL, UNIQUE; e.g. Sanitation, Electricity, Water |
| active | TINYINT(1) | NOT NULL DEFAULT 1 |
| created_at | DATETIME | NOT NULL; UTC timestamp |

### 6.9 audit_logs

| Column | Type | Rules / purpose |
|---|---|---|
| audit_id | BIGINT UNSIGNED | PK, AUTO_INCREMENT |
| actor_user_id | BIGINT UNSIGNED | NULL; FK users.user_id if known |
| action | VARCHAR(100) | NOT NULL; assignment, deadline, role change, etc. |
| entity_type | VARCHAR(50) | NOT NULL; e.g. COMPLAINT, USER |

| | | |
|---|---|---|
| entity_id | BIGINT UNSIGNED | NULL; affected record ID |
| details_json | JSON | NULL; sanitized metadata only, no secrets |
| created_at | DATETIME | NOT NULL; UTC timestamp |

Complaint lifecycle decision: PENDING → ASSIGNED → RESOLVED. A citizen rating of 1 or 2 on a resolved complaint returns it to PENDING for admin triage. Preserve the previous resolution, rating, and status history; do not overwrite the audit trail.

Feedback constraint: one feedback submission per resolution cycle is recommended. Reopened complaints may be resolved and rated again; store feedback records separately so earlier ratings remain available. Admin-only actions and officer actions must be permission-checked on the server.

### 6.10 Relationships and indexes

- users 1-to-many complaints as citizen, assigned/resolved officer, OTP verification records, password-reset records, and audit events.
- complaints 1-to-many attachments, feedback records, and status-history entries.
- complaints reference the citizen user, optional assigned officer, and optional resolving officer.
- departments can be referenced by officers and complaint categories; use a controlled department/category list.
- OTP and password-reset tokens must be stored securely, expire, and be single-use; store a hash of each token where practical.
- audit_logs record privileged actions and workflow changes without storing passwords, OTP values, or JWTs.
- Add indexes for complaints(status, created_at), complaints(officer_id, status), complaints(user_id, created_at), complaints(deadline, status), and all foreign keys.
- Use database transactions when assignment/status/history records or resolution/feedback state changes must succeed together.

## 7. Shared Constants and Naming

| Concept | Canonical name / values |
|---|---|
| Database | civicpulse_db |
| Primary keys | user_id, complaint_id, attachment_id, feedback_id, otp_id, reset_id, history_id, department_id, audit_id |
| Roles | CITIZEN, OFFICER, ADMIN |
| Complaint status | PENDING, ASSIGNED, RESOLVED |
| Priority | LOW, MEDIUM, HIGH |
| Rating | Integer from 1 to 5; ratings 1–2 reopen the complaint |
| Attachment type | BEFORE, AFTER |
| OTP/reset rules | Expiring, single-use, rate-limited; store token hashes where practical |

- Use lowercase snake_case for SQL columns and JSON keys.
- JavaScript variables may use camelCase internally, but API keys stay as documented.
- Use canonical roles CITIZEN, OFFICER, ADMIN; statuses PENDING, ASSIGNED, RESOLVED; priorities LOW, MEDIUM, HIGH.
- Do not rename shared fields or status values without agreement and updating this specification.

## 8. REST API Contract

All Express endpoints use /api/v1. Every endpoint must document authentication, role permissions, request fields, response fields, and errors.

| Method | Endpoint | Purpose / rules |
|---|---|---|
| POST | /auth/register | Create citizen/officer account; validate fields and send email OTP. |

| | | |
|---|---|---|
| | | Admin account creation policy must be restricted. |
| POST | /auth/verify-otp | Validate OTP and set verified=true if valid, unused, and unexpired. |
| POST | /auth/resend-otp | Issue a new OTP with rate limits and invalidate/expire prior codes according to policy. |
| POST | /auth/login | Authenticate verified active user and issue JWT. |
| POST | /auth/logout | Client clears token; if token revocation is implemented, invalidate according to the agreed design. |
| POST | /auth/forgot-password | Start password reset without revealing whether an email exists. |
| POST | /auth/reset-password | Reset password using valid, unexpired, single-use reset token. |
| GET | /auth/me | Return current user's safe profile and role. |
| GET | /users | Admin only: list users with pagination and safe fields. |
| PATCH | /users/:userId/role | Admin only: change role/department; audit the change. |
| PATCH | /users/:userId/active | Admin only: activate/deactivate account; audit the change. |
| POST | /complaints | Citizen only: create complaint with multipart before-image; initial status PENDING. |
| GET | /complaints | Admin only: list/filter all complaints with pagination. |
| GET | /complaints/mine | Citizen only: list the authenticated citizen's own complaints. |
| GET | /complaints/assigned | Officer only: list complaints assigned to the authenticated officer. |
| GET | /complaints/:complaintId | Return complaint details only to permitted roles/owner. |
| POST | /complaints/:complaintId/assign | Admin only: assign eligible officer and set status ASSIGNED. |
| PATCH | /complaints/:complaintId/deadline | Admin only: set/update deadline using YYYY-MM-DD. |
| PUT | /complaints/:complaintId/priority | Admin only: set global priority LOW, MEDIUM, or HIGH. |
| PUT | /complaints/:complaintId/officer-priority | Assigned officer only: update permitted officer priority. |
| POST | /complaints/:complaintId/resolve | Assigned officer only: provide notes and after-image; set status RESOLVED. |
| GET | /complaints/resolved | Admin report view of resolved complaints; apply pagination and permissions. |
| POST | /complaints/:complaintId/feedback | Citizen owner only: submit rating 1–5 and optional text after resolution; rating 1–2 reopens to PENDING. |
| GET | /complaints/:complaintId/history | Return permitted status/action history without secrets. |
| GET | /reports/totals | Admin only: counts grouped by complaint status. |
| GET | /reports/recent | Admin only: recent complaint summary. |
| GET | /reports/daily | Admin only: daily complaint submission statistics. |
| GET | /reports/status | Admin only: status percentages. |
| GET | /reports/officers | Admin only: workload distribution by |

| | | |
|---|---|---|
| | | officer/department. |
| GET | /reports/me/totals | Citizen-only totals for the authenticated citizen's own complaints. |
| GET | /reports/me/recent | Citizen-only recent history for the authenticated citizen. |
| GET | /reports/me/daily | Citizen-only daily submission statistics for their own complaints. |
| GET | /reports/me/status | Citizen-only status breakdown for their own complaints. |
| GET | /reports/export.xlsx | Admin only: export authorized report data to XLSX. |

## 8.1 Standard responses and HTTP codes

Success shape: { success: true, data: ... }. Paginated responses also include pagination.page, pagination.limit, pagination.total, pagination.total_pages. Error shape: { success: false, error: { code, message, details? } }.

| HTTP status | Use |
|---|---|
| 200 | Successful read/update with response |
| 201 | Resource created |
| 400 | Malformed request, invalid rating, invalid status transition, or invalid field |
| 401 | Not authenticated, invalid JWT, or account not verified |
| 403 | Authenticated but role/ownership does not permit action |
| 404 | Complaint/user not found or not visible |
| 409 | Conflict such as duplicate email or invalid workflow state |
| 413 | Upload exceeds configured size limit |
| 415 | Unsupported or invalid upload media type |
| 429 | Rate limit reached for login, OTP, reset, or other protected actions |

## 8.2 Complaint and workflow contract

Complaint endpoints must enforce role and ownership rules. Citizens can create complaints and view their own history; officers can view assigned complaints and resolve them; admins can list all complaints, assign officers, set deadlines/priorities, manage roles, and view system reports.

Use pagination for lists and avoid exposing private email addresses, password hashes, OTPs, reset tokens, or internal secrets in responses.

## 8.3 Complaint response fields

| Field | Type / meaning |
|---|---|
| complaint_id | Identifier |
| title / description | Complaint heading and details |
| category / sub_category | Controlled classification and optional finer category |
| location | Citizen-provided location/address; visibility follows privacy policy |
| status | PENDING, ASSIGNED, or RESOLVED |
| priority | LOW, MEDIUM, HIGH |
| officer_id / deadline | Assigned officer and admin-set deadline, where available |
| image / resolved_image | Before/after evidence storage references, never arbitrary filesystem paths |
| resolved_notes / resolved_at | Officer's resolution notes and timestamp |
| citizen_rating / citizen_feedback | Latest feedback summary; full history kept in feedback records |

# 9. Feedback and Complaint Validation Policy

Validate all complaint descriptions, feedback, categories, priorities, dates, and uploaded images on the server. Feedback is user content; apply length limits, output encoding, rate limits, and report/abuse controls where relevant.

## 9.1 Validation and feedback flow

15. Authenticate the user and verify the complaint exists and is visible to that role.
16. Confirm the citizen owns the complaint before accepting feedback; confirm the complaint is resolved and eligible for rating.
17. Validate rating as an integer from 1 to 5 and limit feedback text length.
18. Validate uploads by size, actual file type, and allowed image formats; generate server-side filenames.
19. Reject invalid transitions, such as an unassigned officer resolving another officer's complaint.
20. For rating 1 or 2, reopen the complaint to PENDING while preserving resolution and feedback history.
21. Return a friendly, clear error message for invalid submissions without revealing internal security details.
22. Use parameterized SQL queries and transactions for complaint, status-history, and feedback writes.
23. Log the action and outcome where appropriate, without logging passwords, OTP values, JWTs, or unnecessary personal data.

| Input type | Expected action |
|---|---|
| Valid complaint description | Allow after authentication, validation, and authorization. |
| Invalid or empty complaint fields | Reject with a clear validation message. |
| Invalid rating outside 1–5 | Reject; do not change complaint status. |
| Rating 1 or 2 on a resolved complaint | Record feedback and reopen to PENDING; preserve resolution and history. |
| Feedback on another citizen's complaint | Reject with 403 or policy-safe error. |
| Resolution attempt by unassigned officer | Reject with 403; no database state change. |
| Expired or reused OTP/reset token | Reject; do not activate account or change password. |
| Excessive OTP/login attempts | Rate-limit and return a safe message. |

- Provide a correction or follow-up path for feedback submitted against the wrong complaint, with an auditable administrative process if implemented.
- Do not allow feedback to bypass authentication, ownership, or the resolved-state requirement.
- Do not store passwords, OTPs, or secrets in complaint descriptions or logs.
- Avoid logging full feedback text unnecessarily; log action, outcome, and identifiers needed for audit.
- Do not return private user or authentication data in public complaint responses.
- Do not expose internal token values or security implementation details in error messages.

# 10. Security and Privacy Requirements

| Area | Required control |
|---|---|
| Authentication | Hash passwords with a maintained library such as bcrypt or Argon2id; never store plaintext. |
| OTP and reset | Use expiring, single-use tokens; rate-limit requests and attempts; never log token values. |
| Authorization | Check role, ownership, and assigned-officer permissions on every protected endpoint. |
| SQL injection | Use parameterized queries; never concatenate user input into SQL. |
| Uploads | Validate actual file type and size, generate server names, restrict paths, and strip public EXIF metadata. |
| JWT | Use a strong secret, sensible expiry, safe storage strategy, and never expose tokens in logs. |
| XSS | Render user text as text, not raw HTML; encode output. |
| CORS | Allow only the agreed frontend origin; do not use wildcard origins with credentials. |
| Rate limits | Limit login, OTP resend/verify, password reset, complaint creation, and uploads. |

| | |
|---|---|
| Audit | Record admin assignments, role changes, deadlines, status changes, resolution, and reopening without secrets. |
| Secrets | Keep real values in .env; commit only .env.example. |

### 10.1 Screenshot policy

A browser-based React app cannot reliably prevent screenshots or screen recordings. Show a notice before complaint submission and avoid promising screenshot prevention. Restrict access to personal records by role and ownership; only public-safe complaint fields should appear in public views.

### 10.2 Public visibility

Complaint descriptions, evidence images, and location details may be sensitive. Tell citizens what is visible to whom. Do not expose private email addresses or authentication data. Limit citizen history to the owning citizen and authorized administrators; officers should see only work they are authorized to handle.

## 11. Image and Location Rules

- Images are supported for before/after evidence; video is deferred.
- Choose a maximum upload size before coding; proposed starting point 5 MB per image, JPEG/PNG/WebP.
- Generate storage names on the server; never trust client-supplied filenames or paths.
- Strip EXIF metadata from public image derivatives where appropriate, and avoid exposing embedded GPS metadata.
- Store attachment metadata in MySQL; keep files in controlled upload storage or object storage.
- Use location only when entered/selected by the citizen for the complaint; do not track the citizen continuously.
- Validate location fields and define whether coordinates, a human-readable location, or both are required for a complaint.
- Do not expose exact personal location beyond the access policy agreed by the team.
- Validate file access permissions and prevent path traversal when serving uploaded evidence.

## 12. Python Service Contract

Python is limited to optional workload/trend analysis and analytics support in this version. It is not required for registration, complaint creation, assignment, resolution, or feedback/reopening.

| Item | Contract |
|---|---|
| Service | FastAPI on http://127.0.0.1:8000 during development |
| Endpoint | POST /api/v1/analyze/workload and/or POST /api/v1/analyze/trends (finalize one contract before implementation) |
| Caller | Express only; protect service with an internal token or equivalent control |
| Input | Bounded counts and time-bucketed complaint fields prepared by Express: complaint_id only if needed, category, status, created_at, deadline state, department/officer workload aggregates. |
| Output | Trend counts and/or workload summaries with time window and analysis_version; no decisions that automatically punish an officer. |
| Restrictions | Do not send passwords, emails, OTPs, JWTs, reset tokens, or unnecessary precise personal location. |
| Failure handling | Core grievance workflow continues if Python is unavailable; optional analytics panel may show unavailable. |

Analytics are descriptive indicators for administrators, not proof of misconduct or objective severity. Show the time range and counting method; core reports should still work without Python.

# 13. Environment and Dependencies

## 13.1 Environment variable names

| Service | Variable names (placeholder values only) |
|---|---|
| Backend | PORT, NODE_ENV, FRONTEND_URL, DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, JWT_SECRET, JWT_EXPIRES_IN, SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, PYTHON_SERVICE_URL, PYTHON_SERVICE_TOKEN, UPLOAD_DIR |
| Frontend | VITE_API_BASE_URL |
| Python | APP_HOST, APP_PORT, PYTHON_SERVICE_TOKEN |

Use .env.example with placeholders. Actual .env files must remain untracked. VITE_ variables are exposed to browser code and must never contain secrets.

## 13.2 Dependency plan

| Module | Proposed dependencies |
|---|---|
| Frontend | React, Vite, React Router, fetch or Axios, a UI/CSS approach, chart library if needed. |
| Backend | Node.js LTS, Express, mysql2, dotenv, cors, helmet, validation library, bcrypt/Argon2 library, JWT library, mail/SMTP library, upload middleware, XLSX export library, test framework. |
| Python | Python 3.11 or another agreed supported version, FastAPI, Uvicorn, Pydantic, pytest. |
| Database | XAMPP MySQL/MariaDB version verified by M3; phpMyAdmin for local administration. |
| Team tools | VS Code, Git, GitHub, npm lockfiles, Python virtual environment. |

Choose and pin compatible versions when implementation begins. Commit package-lock.json and the Python dependency file. Do not commit node_modules, virtual environments, .env, private uploads, email app passwords, JWT secrets, or real user data.

# 14. GitHub Collaboration

Use one repository for all four members, with separate GitHub accounts and separate feature branches.

| Branch | Owner / purpose |
|---|---|
| main | Stable integrated project only; reviewed pull requests. |
| feature/frontend | M1 frontend work. |
| feature/backend | M2 API/security/workflow work. |
| feature/database | M3 schema and seed data. |
| feature/python | M4 Python service and tests. |

24. Repository owner creates the repository and invites the other three members.
25. Everyone clones the same repository and pulls the latest main before starting a task.
26. Each member creates or switches to their assigned feature branch.
27. Make small commits with descriptive messages.
28. Push the branch and open a pull request to main.
29. Review changes and run relevant tests before merging.
30. If shared contracts change, update the specification and tell affected members before merging.
31. Never commit secrets, node_modules, virtual environments, uploaded evidence, or real personal data.

Example commit messages: Add citizen complaint form; Add assignment and deadline APIs; Define complaint schema and history; Add workload analytics tests.

# 15. Folder Structure

```
civicpulse/
├── frontend/                        # M1: React + Vite
│   ├── src/
│   │   ├── components/
│   │   ├── features/
│   │   │   ├── feed/
│   │   │   ├── map/
│   │   │   ├── posts/
│   │   │   ├── comments/
│   │   │   ├── profiles/
│   │   │   └── notifications/
│   │   ├── pages/
│   │   ├── services/                # API client; can switch to mocks
│   │   ├── mocks/                   # Fake responses for standalone UI testing
│   │   ├── constants/
│   │   └── App.jsx
│   ├── tests/
│   │   ├── components/
│   │   ├── features/
│   │   └── fixtures/
│   ├── public/
│   ├── .env.example
│   ├── package.json
│   └── README.md
├── backend/                         # M2: Node.js + Express
│   ├── src/
│   │   ├── config/
│   │   ├── routes/
│   │   ├── controllers/
│   │   ├── services/
│   │   ├── repositories/            # All database access stays here
│   │   ├── middleware/
│   │   ├── validators/
│   │   └── app.js
│   ├── tests/
│   │   ├── unit/
│   │   ├── api/
│   │   ├── fixtures/
│   │   └── mocks/
│   ├── server.js
│   ├── .env.example
│   ├── package.json
│   └── README.md
├── python-service/                  # M4: Python + FastAPI
│   ├── app/
│   │   ├── api/
│   │   ├── schemas/
│   │   ├── services/
│   │   └── core/
│   ├── tests/
│   │   ├── unit/
│   │   ├── api/
│   │   └── fixtures/
│   ├── requirements.txt
│   ├── .env.example
│   └── README.md
├── database/                        # M3: MySQL / XAMPP
│   ├── schema.sql
│   ├── seed-dev.sql
│   ├── test-schema.sql
│   ├── test-data.sql
│   └── README.md
```

```
├── docs/
│   ├── PROJECT_SPEC.md
│   ├── API_SPEC.md
│   ├── DATABASE_SPEC.md
│   ├── PYTHON_API_SPEC.md
│   └── TEST_PLAN.md
├── .gitignore
├── README.md
└── CHANGELOG.md
```

# 16. Testing and Acceptance Criteria

| ID | Acceptance check | Expected result |
|---|---|---|
| AT01 | Registration and OTP | Registration creates an unverified account; valid unexpired OTP verifies it; reused/expired OTP fails. |
| AT02 | Login and roles | Verified user can log in; JWT is validated; each role is restricted to permitted actions. |
| AT03 | Create complaint | Citizen complaint is saved as PENDING with correct owner and validated before-image metadata. |
| AT04 | Admin assignment | Admin assigns an eligible officer, sets priority/deadline, and status becomes ASSIGNED with history. |
| AT05 | Officer resolution | Only assigned officer resolves; notes and after-image are stored; status becomes RESOLVED. |
| AT06 | Feedback rating 1–2 | Feedback is stored; complaint returns to PENDING; prior resolution and history remain. |
| AT07 | Feedback rating 3–5 | Feedback is stored and complaint remains RESOLVED, subject to agreed finalization policy. |
| AT08 | Authorization | Citizen cannot read another citizen's private history or submit feedback for another person's complaint. |
| AT09 | Reports | Status totals, daily counts, recent complaints, and officer workloads match seeded test data. |
| AT10 | Upload security | Oversized, unsupported, or spoofed file uploads are rejected. |
| AT11 | Python unavailable | Registration, complaint CRUD, assignment, resolution, and feedback still work. |
| AT12 | Independent tests | Each member can test their module without requiring the other three modules to be running. |

Independent testing rule: each member must be able to run and verify their own module without the other three modules running. Every module README must state prerequisites, setup steps, environment variables, how to start the module, and how to run tests. Keep real .env files, credentials, node_modules, virtual environments, and personal data out of Git.

M1 — Frontend (Vite/React): run the UI on localhost:5173. Use mock responses in frontend/src/mocks so feed, map, post, comment, profile, and notification screens can be tested before Express exists. Use component/feature tests and fixed fixtures; do not connect React directly to MySQL.

M2 — Backend (Node.js/Express): test routes and business rules using HTTP/API tests, mocked repositories, and mocked Python responses. Run database integration tests separately against a dedicated test database. The frontend must not be

required for backend tests.

M3 — Database (MySQL/XAMPP): maintain repeatable schema and development seed scripts. Import the schema into a separate civicpulse_test_db, verify keys, constraints, indexes, and relationships, and test that the schema can be recreated from a clean database. Never run destructive tests on real or shared data.

M4 — Python service (FastAPI): test the trending-analysis endpoint using sample JSON fixtures and pytest/FastAPI test-client tests. The service must run and return predictable responses without Express or MySQL. Keep its scope optional to core posting, comments, and map features.

Integration sequence: (1) approve the shared API and database contracts; (2) each member passes standalone tests; (3) test Express against MySQL; (4) connect React to Express; (5) connect Express to the Python service; (6) run end-to-end acceptance tests. Use localhost:5173 for Vite, localhost:5000/api/v1 for Express, localhost:3306 for MySQL, and localhost:8000 for FastAPI unless the team agrees to different ports. Record contract changes in docs and CHANGELOG.md before dependent code is updated.

# 17. Suggested Four-Week Plan

| Week | Focus | Exit criteria |
|---|---|---|
| 1 | Approve spec, finalize SQL/API/status contracts, repository and wireframes. | All agree on names, roles, fields, endpoints, workflow, and security decisions. |
| 2 | Develop modules independently using agreed contracts/mock data. | Frontend skeleton, backend endpoints, schema, Python service on branches. |
| 3 | Integrate OTP, complaint submission, admin assignment/deadline, officer resolution, and feedback/reopening. | Primary grievance journey works end-to-end locally. |
| 4 | Security checks, bug fixes, analytics, exports, documentation and demo rehearsal. | Acceptance tests pass and demo is repeatable. |

# 18. Presentation Plan

32. Explain the problem: grievance submissions need traceable ownership, deadlines, and feedback.
33. Register a citizen and demonstrate OTP verification and login.
34. Submit a complaint with category, location, and before-image evidence.
35. Show an admin assigning an officer, setting priority, and setting a deadline.
36. Show the officer resolving the complaint with notes and an after-image.
37. Submit a low citizen rating and demonstrate reopening while preserving history.
38. Show workload/status reports and explain the optional Python analysis.
39. Explain the database, REST API, security controls, and four-member GitHub workflow.

# 19. Decisions to Confirm Before Coding

| Decision | Recommended default | Status |
|---|---|---|
| Authentication | JWT with short expiry and documented storage/expiry strategy; never store tokens in logs. Team must decide whether refresh tokens are needed. | Pending |
| Email OTP | Expiry, single use, resend/attempt | Pending |

| | | |
|---|---|---|
| | limits, and safe error messages. | |
| Roles | CITIZEN, OFFICER, ADMIN; admin account creation must be restricted. | Pending |
| Complaint states | PENDING → ASSIGNED → RESOLVED; rating 1–2 reopens to PENDING without deleting history. | Pending |
| Feedback | One record per resolution cycle; preserve previous feedback when reopened. | Pending |
| Photos | Before/after images; proposed max 5 MB; JPEG/PNG/WebP; validate and strip EXIF from public copies. | Pending |
| Officer assignment | Admin assigns an active officer in the relevant department; assigned officer resolves only their own tickets. | Pending |
| Deadline behavior | Admin sets/edits deadline; overdue reporting shows lateness but does not auto-resolve complaints. | Pending |
| Python analytics | Optional workload/trend summaries only; no core workflow dependency. | Pending |

# 20. Final Agreement

CivicPulse Hub v1.0 is a role-based grievance management and feedback redressal system using React + Vite, Node.js + Express, MySQL/XAMPP, and an optional FastAPI analytics service. Implementation should begin only after the team approves the roles, workflow, database schema, API contracts, security rules, and independent testing plan.

This specification is a draft for team review. Record agreed changes in docs and CHANGELOG.md before dependent implementation work proceeds.